

The Intelligent Network of the Future

Empirical Proof of the HEAIN Core Protocol: An Intelligent Core for Edge AI That Is Resilient, Flexible, Infinitely Scalable, Secure, and Sovereignty-Respecting

Jakkrit Boonma

Independent Researcher · jaggid.boonma@yahoo.co.th · +66 081-007-4833

Date: September 29, 2026

Development: heain-core (Go 1.22), HP Z840 Workstation

Repository: github.com/JakkritB/heain-core

Abstract

Hybrid Edge AI Node [1][2] was introduced as a theoretical framework addressing cloud latency and data-sovereignty concerns through a four-layer fractal governance structure and seven core algorithms. While the normative specification in HEAIN Core Protocol (HEAIN-CORE-1.0) [3][4] went on to fix the rules and wire-level communication formats, both documents remained conceptual frameworks and theoretical specifications whose results had never been proven on real hardware. This paper reports the development and empirical evaluation of HEAIN-CORE-1.1, which builds on HEAIN-CORE-1.0 by carrying it into real Go code running on an HP Z840 workstation, with a focus on verifying that all seven core algorithms (P1–P7) function correctly under real HTTP and mTLS network traffic. The results not only confirm that the protocol operates correctly on physical hardware, but also surface and close four extension mechanisms necessary for production-grade deployment: Duty Profile, Role Ceiling, Escalation Saturation, and P2 Candidate-Pool Discovery. This work therefore serves as the critical bridge proving that the HEAIN Core Protocol can drive an Edge AI network that is resilient, flexible, infinitely scalable, secure under a Zero-Trust model, and fully respectful of data sovereignty in practice.

Keywords: *HEAIN Core Protocol, Hybrid Edge AI Node, Empirical Implementation, Fractal Governance, Escalation Saturation, Live Verification, Edge Computing*

1. Introduction

1.1 Motivation

Hybrid Edge AI Node [1][2] is an architecture paper that unifies six previously separate research threads — hierarchical edge orchestration, hierarchical federated aggregation, swarm intelligence, distributed leader election, human-in-the-loop AI governance, and sovereign edge computing — into a single, organizationally scalable architecture. That paper answers how a system should be designed, but not how to build one that interoperates at the wire level. HEAIN Core Protocol (HEAIN-CORE-1.0) [3][4] was written as a companion, RFC-2119-style specification [5] to close that gap, defining the Envelope format, the `heain://` node addressing scheme, and the governance rules for all seven core procedures. However, HEAIN Core Protocol [3][4] itself remains a specification without a complete reference implementation — its own stated reference implementation covers only a partial subset of Procedures 1–3. The `heain-core` project was started to build a complete, seven-procedure implementation and to prove it through real-hardware testing rather than pseudocode or unit tests alone.

1.2 Gaps Found During Implementation

Before development began, a systematic gap audit was performed between Hybrid Edge AI Node [1][2] and HEAIN Core Protocol [3][4]. At least six mechanisms named or implied by the first paper were never translated into an implementable requirement by the second. Development of heain-core proceeded against this gap list, with every scope reduction from what the two papers might otherwise imply recorded explicitly. This version of heain-core, presented here, covers only the mechanisms that are actually coded and live-verified on real hardware. Any mechanism not yet implemented, or implemented only in part, is explicitly named in its own section as scope deliberately deferred to a future version, never an overlooked gap.

1.3 Scope of This Paper

This paper covers Layer 2 (HEAIN Core Protocol [3][4]) only; Layer 3 (application-specific modules such as heain-job) remains at the planning stage and is out of scope. The remainder of the paper presents the canonical architecture, the seven core algorithms with their real implementation status, the extension mechanisms found necessary during development, the verification methodology, overall results, open limitations, and future work.

1.4 Three-Layer AI Positioning

To avoid any ambiguity about where the system's "intelligence" actually resides, the positioning of AI is separated explicitly into three layers. Layer 1 is Hybrid Edge AI Node [1][2] itself — the architecture that determines where computation happens and how authority is structured across tiers. Layer 2 is the HEAIN Core Protocol [3][4], the subject of this paper, which provides infrastructure-level intelligence: cross-node knowledge exchange via swarm broadcast (P7), self-healing role succession (P6), and policy-boundary reasoning (P5). Layer 3 is application-level AI, delivered through Application Profile modules such as heain-job, heain-access, heain-image, heain-sound, and heain-video — each invoked from within a single protocol procedure, typically P3 (Secure Execution & Staging). In short, the protocol carries the work; the Application Profile modules supply the model. This layering is outside the implementation scope of this paper (see 1.3), but is stated here to position heain-core clearly within the system as a whole.

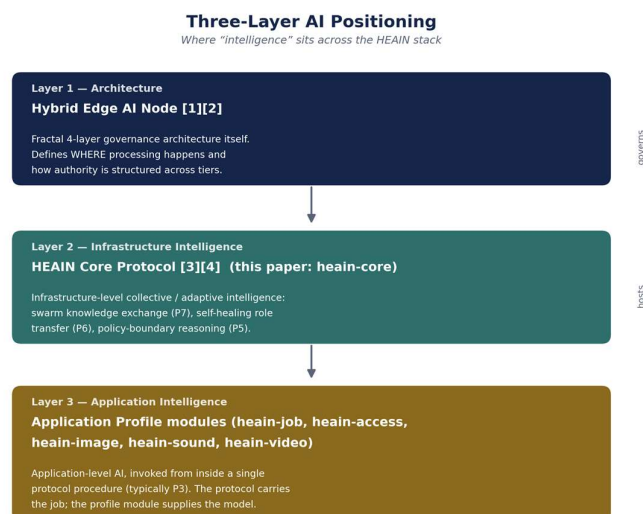


Figure 1 — Three-Layer AI Positioning within the HEAIN Architecture

2. Canonical Architecture

Hybrid Edge AI Node's architecture [1][2] is defined across four layers, three fixed and one that scales with the organization (Table 1, corresponding to Table 2 of [1][2]):

Layer	Function	Scales with org. size?
Management & Orchestration	Hosts the Policy layer; job scheduling, API gateway, security & key management	Yes — see 2.1
Edge Execution	Zero-Trust, in-memory task execution on Worker Nodes	No
Swarm Intelligence	Cross-node knowledge exchange	No
Network & Security	mTLS transport, encryption in transit and at rest	No

2.1 N-tier Generalization

Real implementation found the fixed four-value Tier enum specified by HEAIN Core Protocol [3][4] (Worker, Zone, Secondary, Global Primary) too restrictive for organizations with differing hierarchies. Tier was therefore changed to a free-form descriptive label, and eligibility for promotion was moved to the caller assembling the sibling pool rather than enforced by comparing tier names in shared code — a decision recorded in `design-notes/n-tier-generalization.md` [8]. This change was implemented in `internal/protocol/roletransform` and is `build/test green` (commit `efa30d5`).



Figure 2 — The four-layer canonical architecture and heain-core's real build status

3. The Seven Core Algorithms (P1–P7)

Figure 3 summarizes how the seven core algorithms work together and the data paths between them; it is the primary reference figure for both this paper and academic or business presentations:

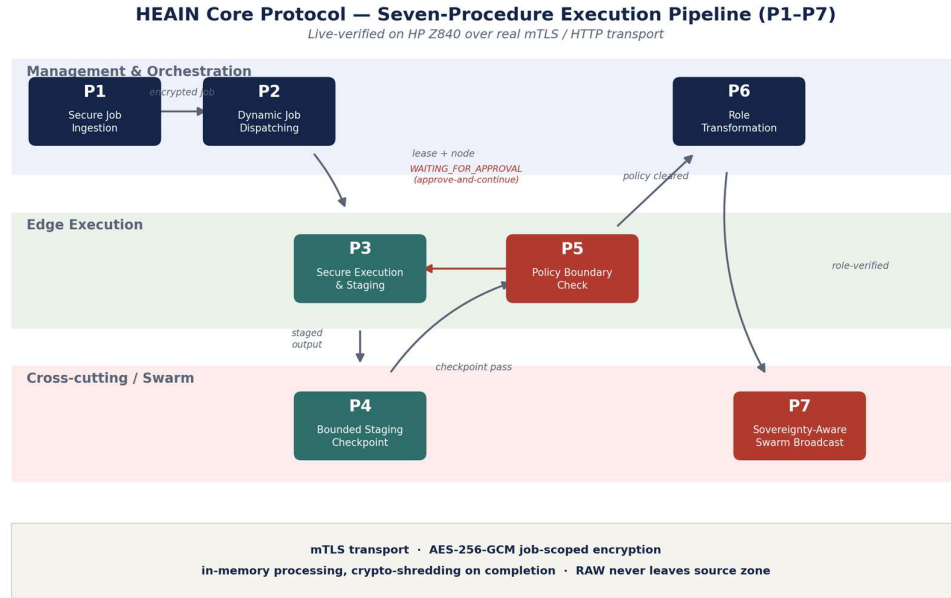


Figure 3 (Primary) — The HEAIN Core Protocol mechanism across procedures P1–P7

Table 2 summarizes the seven core algorithms as specified by Hybrid Edge AI Node [1][2] (corresponding to Table 5 of [1][2]) and operationalized by HEAIN Core Protocol [3][4] as procedures HEAIN-P1 through P7:

#	Algorithm	Layer
P1	Secure Job Ingestion	Management & Orchestration
P2	Dynamic Job Dispatching	Management & Orchestration
P3	Secure Execution & Staging	Edge Execution
P4	Bounded Staging Checkpoint	Edge Execution (background)
P5	Policy Boundary Check	Cross-cutting
P6	Role Transformation	Orchestration + Edge Execution
P7	Sovereignty-Aware Swarm Broadcast	Swarm Intelligence

3.1 P1 — Secure Job Ingestion

P1 was implemented as a new internal/protocol/ingestion package (closing gap 1 named in Section 1.2) — the moment a raw payload is ingested, P1 issues a job-scoped key from a pluggable KMS (AES-256-GCM via an in-memory LocalKMS for this version) and encrypts the raw payload immediately, before building the Envelope and publishing it to a BoltDB-backed DurableQueue, wired into a real /ingest endpoint. Live hardware testing (2026-09-27) confirmed that plaintext never appears in the queue file on disk, not even momentarily. The known limitation is that job classification is caller-supplied rather than auto-derived from the P5 Policy gate, as [3][4] describes.

3.2 P2 — Dynamic Job Dispatching

P2 was implemented with a CapacityProbe that queries real RAM/GPU resources and a BoltDB-backed LeaseStore preventing duplicate assignment. Candidate ranking is deterministic rather than AI-assisted, and was live-verified over real mTLS on 2026-09-27. It was later extended with automatic candidate-pool discovery (Section 4.4).

3.3 P3 — Secure Execution & Staging

P3 was implemented as a SecureExecutor performing decrypt→execute→wipe→route over a real /execute endpoint, using a passthroughExecutor placeholder in place of Layer 3 logic that does not yet exist. Both delivery paths were live-verified on 2026-09-27.

3.4 P4 — Bounded Staging Checkpoint

P4 was implemented as BoundedStaging, disposing of data via crypto-shredding through the KMS while always checking legal hold and minimum retention, plus a Sweep() routine for tier migration and escalation flagging. Both intervals are configurable via BoundedStaging's HotThreshold/EscalateThreshold fields, not hardcoded; the defaults exercised in live testing were 48 hours for tier migration and 30 days for escalation. Live testing found and fixed two distinct disposal idempotency bugs.

3.5 P5 — Policy Boundary Check

P5 implements three fast paths plus a WAITING_FOR_APPROVAL path. A critical gap found live was the absence of any completion path after approval, closed by an approve-and-continue mechanism, live-verified for P7 (2026-09-27) and for P4 over the full P1→P2→P3→P4 pipeline (2026-09-28).

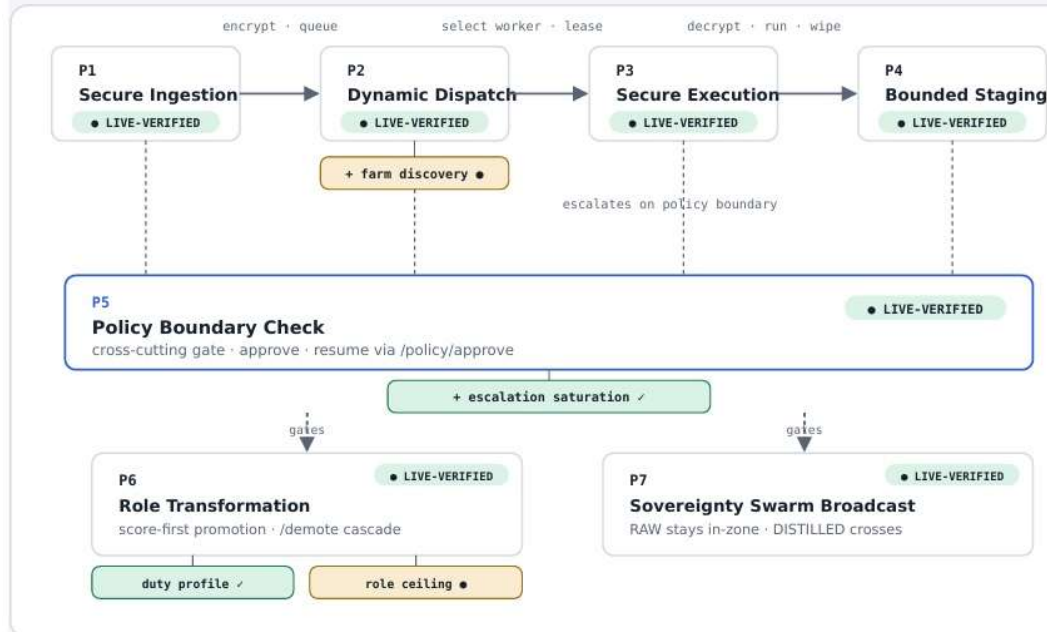
3.6 P6 — Role Transformation

P6 covers split-brain-safe promotion, a /demote reclaim protocol extended to cascade an entire promoted group, acting-role persistence across restarts, and Duty Profile. Every mechanism was live-tested on real hardware, and real bugs were found and fixed along the way.

3.7 P7 — Sovereignty-Aware Swarm Broadcast

P7 guarantees that RAW data never leaves its origin zone; only DISTILLED data may cross zone boundaries, with a mandatory EmbeddingGuard blocking dense-vector payloads. It was wired into a real node network and live-verified end-to-end on 2026-09-27.

Core algorithms — job lifecycle, gated by governance



P1→P4 is the job lifecycle; every stage can escalate through P5, which alone can approve and resume a parked action. P6 and P7 are governance/swarm mechanisms P5 also gates. Chips below a box mark extensions built on top of that mechanism: ✓ live-verified, ● build-verified with live test still pending.

● live-verified on real hardware ● build-verified, live functional test still pending

Figure 4 — The real P1-to-P4 job-delivery pipeline on HP Z840 hardware

4. Extensions Found Necessary During Implementation

Implementing the seven core algorithms surfaced four extension mechanisms that Hybrid Edge AI Node [1][2] only named in principle, or that HEAIN Core Protocol [3][4] never translated into an implementable requirement at all. Every scope decision made for these mechanisms is recorded in design-notes/n-tier-generalization.md [8] and claude/progress-log.md [9].

4.1 Duty Profile

A capacity-gated mechanism attached to each role slot in the Zone Registry. Implemented and live-verified on real hardware (2026-09-26).

4.2 Promotion Scope / Eligible Role Ceiling

Closes a security gap named in HEAIN-SD-1.0 (part of [3][4]): an EDGE-class node must be restricted to the WORKER role only. Reduced to a same-tier equality check since the codebase has no multi-level promotion chain yet. Build-verified (commit 4d018a5, 2026-09-28); live functional verification pending.

4.3 Escalation Saturation Governance-Cadence Safeguard

Hybrid Edge AI Node [1][2] names the risk that operators tend to widen automation thresholds to reduce their own workload, and calls for review on a fixed governance cadence separate from operational tuning — but designs no concrete mechanism. HEAIN Core Protocol [3][4] never mentions it either. heain-core designed and implemented this mechanism from scratch; both steps are live-verified (commit 4d018a5, 2026-09-28).

4.4 P2 Candidate-Pool Discovery via Zone Registry

Closes the gap where every /dispatch caller had to supply its own full candidate list, with no mechanism for a node to discover its own Worker farm automatically. This version uses a static, operator-declared fallback list. Build-verified (commit 4d018a5, 2026-09-28); live functional verification pending.

BEYOND THE BASE PROTOCOL			
Extensions layered on top of P1-P7			
MECHANISM	ATTACHES TO	WHAT IT DOES	STATUS
Duty Profile	P6	Gates which Application-AI duties a promoted node can serve, based on its real capacity — soft, post-selection degradation.	● live-verified
Promotion Scope / Role Ceiling	P6	Hard, pre-selection exclusion — a node whose ceiling equals its own tier never becomes a promotion candidate (e.g. an Edge Node).	● build-verified, live test pending
Escalation Saturation	P5	Governance-cadence tracker — flags when approval thresholds are widened too often or go unreviewed past a set cadence.	● live-verified
P2 Farm-Candidate Discovery	P2	Static, operator-declared Worker farm list (- farm-worker-addr/-ids) that /dispatch falls back to when a caller omits candidates.	● build-verified, live test pending
heain-core · Go 1.22 · target: HP Z840 workstation · design basis: design-notes/n-tier-generalization.md & claude/progress-log.md, 2026-09-28.			

Figure 5 — Summary of the four extension mechanisms and their real verification status

5. Implementation & Verification Methodology

This project held to a two-tier verification standard throughout — stricter than [1][2]'s own acknowledgment that its algorithms remain unbenchmarked pseudocode:

- Build-verified — every change must pass `go build ./...`, `go vet ./...`, and `go test ./...` against the full real codebase.
- Live-verified — every significant mechanism must be exercised against a real running node on real hardware (HP Z840) over real HTTP/mTLS traffic.

This standard surfaced several real defects that unit tests alone would not have caught, including a disposal-idempotency bug in P4 and a stale-status-reporting bug in P6.

6. Overall Results

#	Algorithm	Status	Real Test Result
P1	Secure Job Ingestion	Complete	Confirmed no plaintext left in queue file
P2	Dynamic Job Dispatching	Complete	Selected worker from real capacity data
P3	Secure Execution & Staging	Complete	Execute-and-wipe verified on both delivery paths
P4	Bounded Staging Checkpoint	Complete	Real crypto-shred; 2 idempotency bugs found & fixed
P5	Policy Boundary Check	Complete	Real approve-and-continue resume confirmed
P6	Role Transformation	Complete	Promotion/demote/duty profile verified live
P7	Sovereignty-Aware Swarm Broadcast	Complete	RAW/DISTILLED zone separation confirmed correct

7. Limitations and Open Items

- The AI Reasoning Core is not implemented — P2 always uses a deterministic fallback.
- A full provisioning flow (token, bootstrap, probation, cert-renewal) is not implemented.
- No multi-level promotion chain exists yet, so Promotion Scope reduces to a same-tier equality check.
- The Zone Registry remains a static list, not a dynamic self-registration mechanism.
- Backup/disaster-recovery is explicitly deferred to future development work, out of scope for this v1.

8. Future Work

- Begin a proof-of-concept for heain-job (Layer 3).
- Complete live functional verification for Promotion Scope and P2 Candidate-Pool Discovery.
- Design a fully dynamic Zone Registry once another mechanism genuinely needs it.

9. Conclusion

The work presented here confirms that the seven core algorithms designed in theory by Hybrid Edge AI Node [1][2], and operationalized as a normative specification by HEAIN Core Protocol (HEAIN-CORE-1.0) [3][4], can be implemented and operate correctly on real hardware over real HTTP/mTLS traffic. It further identifies and closes four mechanism-level gaps necessary for real deployment but left unimplemented by either document. HEAIN-CORE-1.1 thereby serves as empirical evidence for the implementability of [1][2][3][4], and proposes the extension mechanisms that should be folded into the formal protocol specification in its next revision.

References

- [1] Boonma, J. "Hybrid Edge AI Node: A Fractal Governance Architecture for Sovereignty-Aware Distributed AI Processing." Zenodo, 2026. DOI: 10.5281/zenodo.22851257
- [2] Boonma, J. "Hybrid Edge AI Node: สถาปัตยกรรมการกำกับดูแลแบบยืดหยุ่นสำหรับการประมวลผล AI แบบกระจายศูนย์ที่เคอร์พอร์ปไตซ์ข้อมูล." Zenodo, 2026. DOI: 10.5281/zenodo.22851587
- [3] Boonma, J. "HEAIN Core Protocol: A Normative Specification for Collective Intelligence in Fractal Edge AI Governance." Zenodo, 2026. DOI: 10.5281/zenodo.22858845
- [4] Boonma, J. "HEAIN Core Protocol: ข้อกำหนดเชิงบรรทัดฐานสำหรับปัญญาร่วมในการกำกับดูแล AI แบบยืดหยุ่นที่ขอบเครือข่าย." Zenodo, 2026. DOI: 10.5281/zenodo.22858994
- [5] Bradner, S. "Key words for use in RFCs to Indicate Requirement Levels." IETF RFC 2119, 1997.
- [6] Ongaro, D., and Ousterhout, J. "In Search of an Understandable Consensus Algorithm (Raft)." USENIX Annual Technical Conference, 2014.
- [7] Boonma, J., and Rujipattanapong, S. "On the Study of the Effectiveness of iSCSI Protocols on the Representative Applications." Proceedings of ITC-CSCC 2007, Busan, Korea, July 8–11, 2007, pp. 979–980.
- [8] Boonma, J. "design-notes/n-tier-generalization.md." heain-core repository (internal design note), 2026.
- [9] Boonma, J. "claude/progress-log.md." heain-core repository (internal implementation and verification log), 2026.

Author: Mr. Jakkrit Boonma | Email: jaggid.boonma@yahoo.co.th | Tel: +66 081-007-4833

© 2026 Jakkrit Boonma. All Rights Reserved. Ownership and copyright in this work belong to Mr. Jakkrit Boonma from the date of public disclosure, under applicable copyright law.